Spiroscopic Recursive Transformers

Learned function-space neighborhoods for bounded program discovery

Morality Lab Research Note - revised August 2026

Abstract

Spiroscopic Recursive Transformers (SRTs) are bounded proposal mechanisms that traverse a learned neighborhood of related functions before a conventional verifier chooses an exact result. Earlier SRT studies were predominantly negative: phase-structured loops often reduced to scan-and-pool, lost to simple baselines, or missed preregistered confirmation gates. This paper reports a narrower positive result in object-centric, ARC-like program discovery. A Tiny Recursive Model (TRM) proposed one rigid program; an SRT generated nearby program rewrites in a train-only learned coordinate system; and a deterministic LDT stage executed every candidate and accepted only exact matches. On 64 synthetic tasks drawn from held-out operator pairs, composition motifs, and program families, the SRT pipeline solved 47/64 tasks (73.44%) versus 35/64 (54.69%) for the strongest matched residual edit beam, a difference of 18.75 percentage points. The root-block bootstrap interval was [4.69, 34.38] percentage points, and gains appeared in all four held-out families. The same-set shuffled-order control also solved 47/64, so the evidence supports candidate-set coverage, not useful candidate ordering or phase dynamics. Direct top-k search solved 0/64, graph search 22/64, random and Sobol search 1/64 each, and exact target-program reachability was 64/64. A descriptive run on 24 public ARC tasks solved none. The result is therefore a synthetic mechanism confirmation, not evidence of public ARC capability or a general transformer replacement. It identifies one class of SRT value: preserving semantically useful rewrites within a small execution-verified search budget.

Evidence at a glance

Measure	Observed result
Synthetic exact solve	SRT 47/64 (73.44%); residual beam 35/64 (54.69%)
Matched-control margin	+18.75 points; root-block 95% interval [+4.69, +34.38]
Held-out families	Positive margins in 4/4 composition families
Order counterfactual	Same candidate set, shuffled order: 47/64 (tie)
Public ARC	0/24 exact solves; descriptive only
Current reproducibility	Summary recovered; rows and checkpoints not trustworthy

1. The question after the negative results

Recursive neural systems are often motivated by convergence. Repeated application of a tied map can refine a latent state toward a stable answer. SRTs invert part of that schedule. They first expand into a bounded set of alternatives, then collapse through selection or exact verification. In the notation used throughout this project, a schedule such as D2C3 means two divergent proposal steps followed by three consolidating steps.

The broad hypothesis - that phase-structured divergence improves reasoning - did not survive the first experimental campaigns. Circular pooling worked on origin-invariant rotational targets but not on temporal-origin targets. A learned static atlas failed to beat a raw trace baseline. A feedback-navigator study produced a fresh-root effect of +0.0918 against a frozen +0.10 requirement and was correctly classified as non-confirmatory. Another objective collapsed its labels before training and was closed as an invalid mechanism test.

Those failures sharpen the question. SRT value should not be sought in geometric novelty alone. It should be sought where three conditions hold:

1. One proposal is often close to a correct structured solution but is wrong in a small number of semantically meaningful ways.

2. A small budget of neighboring rewrites can cover those mistakes more efficiently than syntax-local edits or unstructured sampling.

3. A cheap, exact verifier can reject every attractive but incorrect candidate.

Bounded program discovery satisfies these conditions. The hypothesis tested here is correspondingly narrow: a learned coordinate system over program functions can preserve useful semantic neighbors of a rigid proposal under held-out compositions.

2. TRM to SRT to LDT

The tested pipeline separates proposing, expanding, and verifying:

```
input grids -> TRM rigid proposal -> SRT semantic rewrites -> LDT execution -> exact match
```

The TRM stage emits a single program p0 from an object-centric domain-specific language. It is intentionally not allowed to hide extra search inside a large top-k list. The SRT maps p0 into a learned function-space coordinate, visits a bounded neighborhood, and decodes five to ten rewrites. The LDT stage is deterministic: it executes each candidate on every training pair, checks shape and cell equality, and returns a candidate only if all demonstrations match exactly. No verifier score is used as a soft substitute for correctness.

Let E(p, x) be execution of program p on grid x and let D be the set of training input-output pairs. The verifier is

```
V(p; D) = product over (x, y) in D of 1[E(p, x) = y].
```

The SRT does not predict the final grid directly. It proposes a set N_phi(p0, x) learned from training programs, while the LDT enforces V(p; D) = 1. This division of labor is important. It lets the experiment ask whether the neighborhood contains useful functions without conflating neighborhood quality with a learned reward model.

3. Object-centric program space

The synthetic generator uses an object-centric DSL with operations from the following families:

- connected-component and color-based selection;
- translation and conditional component motion;
- recoloring and palette substitution;
- bounding-box cropping;
- repetition and tiling;
- reflection and symmetry completion;
- masking, intersection, and subtraction;

- conditional fills based on object or region properties.

Programs have depth three to five. Tasks are generated from programs, not hand-labeled after inspection. The split holds out structure at three levels:

- complete operator pairs, so the learner cannot merely reuse every adjacent transition;
- composition motifs, so a familiar operator can occur in an unfamiliar local role;
- program families, so evaluation includes coherent classes of unseen compositions.

This is stronger than holding out odd orbit positions, seeds, or surface forms. A phase-parity split can make a designed gear look like generalization while leaving the underlying composition family unchanged. The present split instead asks whether training has induced a neighborhood that remains useful when the program grammar recombines known primitives in excluded ways.

4. What the SRT learns

The SRT neighborhood is learned only from training programs and their execution behavior. It is intended to place programs near one another when they implement related transformations, even when their token-level edit distance is misleading. A translation followed by recoloring may be functionally closer to a conditional translation followed by recoloring than to a one-token deletion that breaks object selection.

The primary neighborhood used the frozen D2C3 policy selected before confirmation. Expansion moves away from the rigid proposal along learned function coordinates. Collapse decodes a small candidate set and removes duplicates. The exact verifier then evaluates those candidates. The experiment does not claim that the coordinates form a globally smooth program manifold. The operational claim is weaker: local moves in this representation preserve useful rewrite hypotheses often enough to improve exact solve rate under a fixed budget.

This distinction matters because the same-set shuffle control preserved every candidate and changed only order. Its tie with the primary SRT shows that the experiment did not detect an advantage from visit order. The supported object is the candidate set produced by the neighborhood, not the sequence in which the orbit visits it.

5. Confirmatory design

The synthetic confirmation used 64 independently generated tasks arranged in eight root blocks and four held-out composition families. The policy, candidate budget, generator, split, matched controls, decision rule, and minimum effect were frozen before the sealed run. The primary gate required at least a 10-point exact-solve gain over the strongest matched search control, a positive root-block confidence interval, gains across at least three held-out families, and no meaningful regression on simple null tasks.

The comparison set was designed to separate semantic coverage from generic extra compute:

- direct TRM top-1 and top-k;
- an edit-distance beam;
- a matched residual rewrite beam;
- graph search over legal program edits;
- random and Sobol sampling;
- a shuffled-neighborhood control;
- a same-candidate-set shuffled-order control;
- exhaustive target-program reachability as a ceiling.

Every method used the same task set and exact executor. Budget-matched controls were restricted to the same number of evaluated candidates wherever their construction permitted. The ceiling is not a practical solver; it checks that the target remains expressible and reachable within the generated DSL.

6. Main result

The primary SRT pipeline solved 47 of 64 tasks, or 73.44%. The strongest matched control, residual beam search, solved 35 of 64, or 54.69%. The absolute difference was 18.75 percentage points, above the registered 10-point threshold.

The eight root-block differences were

```
[+0.125, +0.625, +0.125, 0.000, +0.250, +0.250, +0.250, -0.125].
```

The 95% root-block bootstrap interval was [+0.046875, +0.34375]. Fifteen tasks were SRT-only wins, three were residual-only wins, and 46 were ties. Conditional on the 18 decisive tasks, the SRT win rate was 83.33%, with a Wilson interval of [60.78%, 94.16%].

The family-level gains were:

- mask, fill, and symmetry: +25.0 points;
- conditional component motion: +25.0 points;
- crop, repeat, and recolor: +12.5 points;
- select, translate, and recolor: +12.5 points.

The result therefore crossed the registered mechanism gate and did so across all four held-out families. One root block was negative and one tied, so the gain was not universal at the block level.

7. Controls and the actual source of value

The full comparison makes the boundary clearer. Direct TRM top-1 and top-k solved 0/64. The token edit beam also solved 0/64. Graph search solved 22/64 (34.38%). Random and Sobol search each solved 1/64 (1.56%). Exhaustive target-program reachability and the oracle-at-budget check were both 64/64.

The residual beam is the important control because it received a comparable rewrite budget and was substantially stronger than graph, random, Sobol, or token-local search. The SRT margin over that control is evidence that the learned neighborhood retained useful program variations.

However, the exact same candidate-set shuffle also solved 47/64. That result rejects the stronger story that a spiroscopic visitation order is itself responsible. Candidate-set hashes were preserved, and changing order did not degrade exact solve rate. The positive result is thus representation-and-coverage value: the SRT machinery generated a better bounded set. It is not evidence for order-sensitive holonomy, phase intelligence, or an advantage of orbit chronology.

This is a meaningful narrowing rather than a nullification. Many small-model skill flows need only a compact proposal set that a reliable tool can verify. They do not require the proposal generator's internal sequence to have causal importance. But the distinction should control how the result is described and what is tested next.

8. Nulls and external validity

Novelty did not hurt the generated simple-task nulls. On that lane, SRT exact solve was 68.75% versus 50.00% for residual beam. Repeat-flip rate, position sensitivity, and hard-fail selection were all zero; deterministic coverage was 100%. These checks argue against the gain being purchased by indiscriminate disruption of easy cases.

The public ARC lane was deliberately descriptive and ran only after the synthetic gate. It solved 0 of 24 selected public tasks. This is the most important external-validity result. The synthetic DSL and learned neighborhood did not transfer into a practical public ARC solver. The public run cannot support a capability claim, and the synthetic margin should not be translated into an expected ARC-AGI score.

ARC exact-match tasks combine perception, object discovery, representation choice, program induction, and execution. The synthetic study isolates the last part of that stack after object-centric structure has already been made available. A 0/24 result is consistent with a neighborhood that helps once the right program language and proposal are in hand but does not solve representation discovery.

9. Relationship to the earlier SRT program

The bounded-program result changes, but does not erase, the earlier record. The torque atlas, static atlas, feedback navigator, and failed label mechanism remain negative or non-confirmatory. Their controls prevented a visually appealing trajectory from being misread as reasoning. The new experiment succeeds because it asks a different and more constrained question.

Earlier studies often lacked an exact downstream criterion or used tasks on which pooling could absorb the alleged phase structure. Here, candidates denote executable programs, exact verification is available, and held-out composition families make semantic rewrite quality consequential. This is precisely the kind of setting in which an SRT can add value without being a general model replacement.

The combined evidence supports a sparse map of utility:

- not broad gains from divergence;
- not reliable order-sensitive computation;
- not direct public ARC solving;
- positive bounded candidate coverage in an execution-verifiable compositional program space.

10. Implications for small-model skill flows

The result suggests a practical architecture for small models that cannot reliably complete a structured task in one pass. The small model proposes one rigid artifact. An SRT-like neighborhood generator expands only along learned semantic axes. A deterministic tool checks each candidate. An audit layer records the winning program, execution trace, and rejected alternatives.

This pattern can apply beyond grids when three properties are present: a compact structured output, a local family of plausible repairs, and a verifier cheaper than generating another large batch from scratch. Candidate domains include query plans, schema mappings, short formal proofs, configuration repair, bounded code transformations, and storyworld effect packets with explicit state variables.

For storyworlds, the interface should remain structured. An action option can be paired with a vector of estimated changes to named variables, uncertainty bounds, and a neighborhood identifier. The SRT proposes nearby effect models; the TRM or language model chooses; the simulator or rule engine verifies consequences where possible. The ARC-like result does not establish value in that domain, but it gives a concrete hypothesis: semantic neighborhoods may help when choices fail through a small number of structured, repairable effect estimates.

11. Limitations and recovery status

The synthetic result was recorded by the completed experiment and evaluator audit, but a later filesystem incident destroyed or contaminated the underlying row files and checkpoints. The surviving recovery contains the exact result summary and task-log reconstruction, including solve counts, block differences, family margins, control outcomes, and the operational audit decision. It does not contain a trustworthy runnable checkpoint or intact per-task row set.

This creates two claim layers:

1. Historical result: the sealed run was recorded as passing the synthetic gate with the statistics reported above.
2. Present reproducibility: the recovered repository cannot independently regenerate or fully audit that run from its surviving artifacts.

Accordingly, the result should be treated as evidence from a completed but currently unreproducible internal experiment. It should not be used as a benchmark submission or a final empirical claim until the implementation is reconstructed and the confirmation is rerun from frozen manifests. The public ARC result, also recovered as 0/24, reinforces the need for restraint.

Other limitations are intrinsic rather than forensic. The benchmark generator and DSL encode an object-centric prior. Exact verification makes false acceptance unusually cheap to avoid. The test set has only eight root blocks. The best control may still omit stronger program-synthesis methods. The same-set shuffle tie leaves the spiroscopic ordering hypothesis unconfirmed. Finally, no result here establishes scaling behavior, sample efficiency relative to large pretrained models, or robustness outside the generated grammar.

12. Required replication

The next study should reconstruct the evaluator before changing the model. It should publish or preserve:

- the full DSL definition and generator commit;
- train, discovery, and confirmation manifests;
- all held-out operator pairs, motifs, and families;
- the frozen D2C3 neighborhood policy and candidate budget;
- candidate-set hashes for every method;
- per-task programs, execution traces, and exact outcomes;
- root-block bootstrap code and confidence interval;
- simple-null and same-set shuffle reports;
- an independently generated confirmation seed set.

Replication should compare the SRT neighborhood against a stronger learned rewrite model, retrieval from training programs, minimum-description-length search, and a neural-guided enumerator. The primary question is whether semantic coverage remains better under matched execution budgets. Order sensitivity should be a separate gate, requiring a preregistered loss when the identical candidate set is permuted.

Public ARC should remain a descriptive lane until the system includes learned object extraction and task-conditioned DSL induction. A failure there should not invalidate a verified synthetic mechanism, but neither should a synthetic mechanism be advertised as ARC solving.

13. Conclusion

The SRT research program produced mostly negative results and one narrow positive result. In bounded object-centric program discovery, a learned function-space neighborhood expanded a single rigid proposal into a candidate set that outperformed the strongest matched residual search control by 18.75 percentage points on held-out synthetic compositions. The interval was positive and gains spanned four families. Yet candidate order contributed nothing detectable, public ARC performance was 0/24, and the surviving artifacts cannot currently reproduce the run.

The defensible conclusion is therefore specific: SRT-style learned neighborhoods may add value as compact proposal generators inside verifier-backed small-model skill flows. The evidence does not support general recursive intelligence, public ARC competence, or causal value from spiral ordering. Rebuilding and rerunning the sealed experiment is the next scientific obligation.

References

Chollet, F. On the Measure of Intelligence. arXiv:1911.01547, 2019.

Ellis, K., Wong, C., Nye, M., Sable-Meyer, M., Cary, L., Morales, L., Hewitt, L., Solar-Lezama, A., and Tenenbaum, J. B. DreamCoder: Growing Generalizable, Interpretable Knowledge with Wake-Sleep Bayesian Program Learning. arXiv:2006.08381, 2020.

Jolicoeur-Martineau, A. Less is More: Recursive Reasoning with Tiny Networks. arXiv:2510.04871, 2025.

Morality Lab. Spiroscopic Recursive Transformers experiment record and recovery audit. Internal research artifacts, August 2026.